

ESP32 Bluetooth RC Control System

Miguel Chip

January 29, 2026

Abstract

This document describes the design and implementation of a Bluetooth-based RC controller using the ESP32 microcontroller.

Contents

1	Introduction	2
2	System Overview	3
3	Communication Protocol	4
3.1	Packet Structure	4
4	Main Program	5
4.1	Initialization	5
4.2	Main Loop	5
5	Conclusion	25
A	Full Source Code	26

Chapter 1

Introduction

This project controls RC vehicles using Bluetooth communication.

Chapter 2

System Overview

The ESP32 receives control packets and drives motors using PWM.

Chapter 3

Communication Protocol

3.1 Packet Structure

Listing 3.1: RC Packet Structure

```
typedef struct __attribute__((packed)) {  
    byte startByte1;  
    byte startByte2;  
    uint16_t leftStickX;  
    uint16_t leftStickY;  
    uint16_t rightStickX;  
    uint16_t rightStickY;  
    byte checksum;  
} RcPacket;
```

Each field represents a control channel.

Chapter 4

Main Program

4.1 Initialization

Listing 4.1: ESP32 setup function

```
void setup() {  
    Serial.begin(115200);  
    initBluetooth();  
}
```

4.2 Main Loop

Listing 4.2: Main loop

```
void loop() {  
    if (receivePacket()) {  
        applyControls();  
    }  
}
```

```
1 void loop() {  
2     if (receivePacket()) {  
3         applyControls();  
4     }  
5 }
```

Listing 4.3: Main loop

```
1 #include "BluetoothSerial.h"  
2  
3 BluetoothSerial SerialBT;  
4 #define DEVICE_NAME "ESP32-BT"  
5  
6 typedef struct __attribute__((packed)) {  
7     byte startByte1;  
8     byte startByte2;  
9     uint16_t leftStickX;  
10    uint16_t leftStickY; // Bytes 4-5  
11    uint16_t rightStickX; // Bytes 6-7  
12    uint16_t rightStickY; // Bytes 8-9  
13    uint16_t leftKnob; // Bytes 10-11 (Was byte)  
14    uint16_t rightKnob; // Bytes 12-13 (Was byte)  
15    byte switches; // Byte 14  
16    byte checksum; // Byte 15
```

```

17 byte endByte1; // Byte 16 (From Android code)
18 byte endByte2; // Byte 17 (From Android code)
19 }
20 RcPacket;
21
22 typedef union {
23 RcPacket data;
24 byte bytes[sizeof(RcPacket)];
25 }
26 StatePacketUnion;
27
28 StatePacketUnion rcStatePacket;
29 const int STATE_PACKET_SIZE = sizeof(RcPacket);
30 typedef struct __attribute__((packed)) {
31 byte startByte1;
32 byte startByte2;
33 byte eventId;
34 byte checksum;
35 }
36 EventPacket;
37
38 typedef union {
39 EventPacket data;
40 byte bytes[sizeof(EventPacket)];
41 }
42 EventPacketUnion;
43
44 EventPacketUnion rcEventPacket;
45 const int EVENT_PACKET_SIZE = sizeof(EventPacket);
46
47 typedef struct __attribute__((packed)) {
48 byte header1; // 0xCC
49 byte header2; // 0x11
50 uint16_t leftPanelValue;
51 uint16_t rightPanelValue;
52 byte panelStates;
53 byte checksum;
54 }
55 PanelPacket;
56 const int PANEL_PACKET_SIZE = sizeof(PanelPacket);
57 PanelPacket panelPacket;
58
59 typedef struct __attribute__((packed)) {
60 byte header1; // 0xCC
61 byte header2; // 0x22
62 byte analogValue;
63 byte batteryLevel;
64 byte checksum;
65 }
66 IndicatorPacket;
67 const int INDICATOR_PACKET_SIZE = sizeof(IndicatorPacket);
68 IndicatorPacket indicatorPacket;
69
70 typedef struct __attribute__((packed)) {
71 byte header1; // 0xCC
72 byte header2; // 0x33
73 byte numPlots; // Number of plot values being sent
74 byte plotValue1; // First plot value

```

```

75 byte plotValue2; // Second plot value
76 byte plotValue3;
77 byte checksum;
78 }
79 PlotPacket;
80 const int PLOT_PACKET_SIZE = sizeof(PlotPacket);
81 PlotPacket plotPacket;
82
83 // --- START OF NEW CONFIGURATION PACKET LOGIC ---
84
85 // 1. Define the names for our plots that we will send to the Android app.
86 const char * plotNames[] = {
87     "Volts",
88     "Amps",
89     "RPMs"
90 };
91 const int NUM_PLOT_NAMES = sizeof(plotNames) / sizeof(plotNames[0]);
92
93 // 2. Create a function to build and send the variable-length config packet.
94 void sendConfigTelemetry() {
95     byte packetBuffer[128]; // A buffer large enough to hold our headers and names
96     int packetIndex = 0;
97
98     // Header for Config Packet: 0xCC 0x44
99     packetBuffer[packetIndex++] = 0xCC;
100    packetBuffer[packetIndex++] = 0x44;
101
102    // Number of names we are sending
103    packetBuffer[packetIndex++] = NUM_PLOT_NAMES;
104
105    // For each name, write its length followed by the name itself
106    for (int i = 0; i < NUM_PLOT_NAMES; i++) {
107        int nameLen = strlen(plotNames[i]);
108        packetBuffer[packetIndex++] = nameLen; // Write the length of the name
109        memcpy( & packetBuffer[packetIndex], plotNames[i], nameLen); // Write the name
110        bytes
111        packetIndex += nameLen;
112    }
113
114    // Note: The simple config parser on Android doesn't require a checksum for
115    this packet.
116
117    if (SerialBT.hasClient()) {
118        SerialBT.write(packetBuffer, packetIndex);
119        Serial.println("Sent Plot Names Configuration Packet.");
120    }
121 }
122
123 // --- END OF NEW CONFIGURATION PACKET LOGIC ---
124
125 unsigned long lastPanelSendTime = 0;
126 const long panelInterval = 1000;
127 unsigned long lastIndicatorSendTime = 0;
128 const long indicatorInterval = 250;
129 unsigned long lastPlotSendTime = 0;
130 const long plotInterval = 50;
131
132 void processAndPrintPacket() {
133     Serial.println("--- Valid 18-Byte State Packet Received! ---");
134 }

```

```

131 Serial.printf("  Sticks L_X: %u, L_Y: %u, R_X: %u, R_Y: %u\n",
132 rcStatePacket.data.leftStickX, rcStatePacket.data.leftStickY,
133 rcStatePacket.data.rightStickX, rcStatePacket.data.rightStickY);
134
135 Serial.printf("  Knobs L: %u, R: %u\n", rcStatePacket.data.leftKnob,
136 rcStatePacket.data.rightKnob);
137
138 bool switch1 = (rcStatePacket.data.switches & (1 << 0)) != 0;
139 bool switch2 = (rcStatePacket.data.switches & (1 << 1)) != 0;
140 bool switch3 = (rcStatePacket.data.switches & (1 << 2)) != 0;
141 bool switch4 = (rcStatePacket.data.switches & (1 << 3)) != 0;
142 bool switch5 = (rcStatePacket.data.switches & (1 << 4)) != 0;
143 bool switch6 = (rcStatePacket.data.switches & (1 << 5)) != 0;
144 Serial.printf("  Switches (Byte: %d): S1=%d, S2=%d, S3=%d, S4=%d, S5=%d, S6=%d\n",
145 rcStatePacket.data.switches, switch1, switch2, switch3, switch4, switch5,
146 switch6);
147 Serial.println("-----");
148 }
149
150 void processEventPacket() {
151 Serial.println("--- Valid 4-Byte Event Packet ---");
152 Serial.printf("  Event ID: 0x%02X\n", rcEventPacket.data.eventId);
153 Serial.println("-----");
154
155 switch (rcEventPacket.data.eventId) {
156 case 0x01:
157 Serial.println("  Action: Top-Left Button Pressed!");
158 break;
159 case 0x02:
160 Serial.println("  Action: Top-Right Button Pressed!");
161 break;
162 case 0x03:
163 Serial.println("  Action: Bottom-Left Button Pressed!");
164 break;
165 case 0x04:
166 Serial.println("  Action: Bottom-Right Button Pressed!");
167 break;
168 default:
169 Serial.println("  Action: Unknown Event ID.");
170 break;
171 }
172 }
173
174 // --- START OF SEPARATE SEND FUNCTIONS ---
175
176 void sendPanelTelemetry() {
177 panelPacket.header1 = 0xCC;
178 panelPacket.header2 = 0x11;
179 panelPacket.leftPanelValue = 1234;
180 panelPacket.rightPanelValue = 5678;
181
182 byte states = 0;
183 states |= (1 << 0); // leftOn = true
184 states |= (1 << 1); // rightOn = false
185 states |= (1 << 2); // leftColorIsGreen = true
186 states |= (0 << 3); // rightColorIsGreen = false
187 panelPacket.panelStates = states;

```

```

186 byte checksum = 0;
187 // Sum payload bytes (from byte 2 up to the checksum byte)
188 for (int i = 2; i < PANEL_PACKET_SIZE - 1; i++) {
189     checksum += ((byte * ) & panelPacket)[i];
190 }
191 panelPacket.checksum = checksum;
192
193
194 if (SerialBT.hasClient()) {
195     SerialBT.write((byte * ) & panelPacket, PANEL_PACKET_SIZE);
196 }
197 }
198
199 void sendIndicatorTelemetry() {
200     indicatorPacket.header1 = 0xCC;
201     indicatorPacket.header2 = 0x22;
202     indicatorPacket.analogValue = 75; // Example static value
203     indicatorPacket.batteryLevel = 98; // Example static value
204
205     byte checksum = 0;
206     for (int i = 2; i < INDICATOR_PACKET_SIZE - 1; i++) {
207         checksum += ((byte * ) & indicatorPacket)[i];
208     }
209     indicatorPacket.checksum = checksum;
210
211     if (SerialBT.hasClient()) {
212         SerialBT.write((byte * ) & indicatorPacket, INDICATOR_PACKET_SIZE);
213     }
214 }
215
216 void sendPlotTelemetry() {
217     plotPacket.header1 = 0xCC;
218     plotPacket.header2 = 0x33;
219     plotPacket.numPlots = 3;
220
221     float time = millis() / 1000.0 f;
222     plotPacket.plotValue1 = (uint8_t)((((sin(time * 1.5 f * PI) * 0.4 f) + 0.6 f) *
223         255));
224     plotPacket.plotValue2 = (uint8_t)((((cos(time * 3.0 f * PI) * 0.2 f) + 0.25 f)
225         * 255));
226     plotPacket.plotValue3 = (uint8_t)((((cos(time * 3.0 f * PI) * 0.2 f) + 0.5 f) *
227         255));
228
229     byte checksum = 0;
230     for (int i = 2; i < PLOT_PACKET_SIZE - 1; i++) {
231         checksum += ((byte * ) & plotPacket)[i];
232     }
233     plotPacket.checksum = checksum;
234
235     if (SerialBT.hasClient()) {
236         SerialBT.write((byte * ) & plotPacket, PLOT_PACKET_SIZE);
237     }
238 }
239
240 void setup() {
241     Serial.begin(115200);
242     SerialBT.begin(DEVICE_NAME);
243     Serial.println("ESP32 Bluetooth Receiver Ready.");

```

```

241 Serial.printf("Listening for State Packets (%d bytes) and Event Packets (%d
    bytes).\n", STATE_PACKET_SIZE, EVENT_PACKET_SIZE);
242 }
243
244 void loop() {
245     // --- START OF MODIFICATION: Send config on new client connection ---
246     static bool clientIsConnected = false;
247     if (SerialBT.hasClient()) {
248         // If a client has just connected, send the configuration.
249         if (!clientIsConnected) {
250             Serial.println("Client Connected!");
251             clientIsConnected = true;
252             delay(200); // Give the client a moment to be ready before sending data
253             sendConfigTelemetry(); // Send the plot names right after connecting
254         }
255         else {
256             // If the client has disconnected, reset the flag.
257             if (clientIsConnected) {
258                 Serial.println("Client Disconnected.");
259                 clientIsConnected = false;
260             }
261         }
262         // --- END OF MODIFICATION ---
263
264         static byte buffer[STATE_PACKET_SIZE * 2];
265         static int bytesRead = 0;
266
267         while (SerialBT.available()) {
268             if (bytesRead < sizeof(buffer)) {
269                 buffer[bytesRead++] = SerialBT.read();
270             } else {
271                 memmove(buffer, buffer + 1, sizeof(buffer) - 1);
272                 bytesRead--;
273                 buffer[bytesRead++] = SerialBT.read();
274             }
275         }
276
277         while (true) {
278             int packetStart = -1;
279             bool isEvent = false;
280             int packetSize = 0;
281
282             for (int i = 0; i <= bytesRead - 2; i++) {
283                 if (buffer[i] == 0xAA && buffer[i + 1] == 0x55) {
284                     if (bytesRead - i >= STATE_PACKET_SIZE) {
285                         packetStart = i;
286                         isEvent = false;
287                         packetSize = STATE_PACKET_SIZE;
288                         break;
289                     }
290                 }
291                 if (buffer[i] == 0xBB && buffer[i + 1] == 0x66) {
292                     if (bytesRead - i >= EVENT_PACKET_SIZE) {
293                         packetStart = i;
294                         isEvent = true;
295                         packetSize = EVENT_PACKET_SIZE;
296                         break;
297                     }

```

```

298 }
299 }
300
301 if (packetStart == -1) {
302     break;
303 }
304
305 if (isEvent) {
306     memcpy(rcEventPacket.bytes, & buffer[packetStart], packetSize);
307     byte eventId = rcEventPacket.data.eventId;
308     byte checksum = rcEventPacket.data.checksum;
309     if (eventId == checksum) {
310         processEventPacket();
311     } else {
312         Serial.println("Event packet checksum mismatch!");
313     }
314 } else {
315     memcpy(rcStatePacket.bytes, & buffer[packetStart], packetSize);
316     byte receivedChecksum = rcStatePacket.data.checksum;
317     byte calculatedChecksum = 0;
318     for (int i = 2; i < STATE_PACKET_SIZE - 3; i++) {
319         calculatedChecksum += rcStatePacket.bytes[i];
320     }
321     if (receivedChecksum == calculatedChecksum) {
322         processAndPrintPacket();
323     } else {
324         Serial.printf("State Checksum mismatch! R: %d, C: %d\n", receivedChecksum,
325             calculatedChecksum);
326     }
327 }
328
329 int bytesToRemove = packetStart + packetSize;
330 if (bytesToRemove > bytesRead) {
331     bytesRead = 0;
332 } else {
333     int bytesRemaining = bytesRead - bytesToRemove;
334     memmove(buffer, & buffer[bytesToRemove], bytesRemaining);
335     bytesRead = bytesRemaining;
336 }
337
338 unsigned long currentTime = millis();
339
340 if (clientIsConnected) {
341     if (currentTime - lastPanelSendTime >= panelInterval) {
342         lastPanelSendTime = currentTime;
343         sendPanelTelemetry();
344     }
345     if (currentTime - lastIndicatorSendTime >= indicatorInterval) {
346         lastIndicatorSendTime = currentTime;
347         sendIndicatorTelemetry();
348     }
349     if (currentTime - lastPlotSendTime >= plotInterval) {
350         lastPlotSendTime = currentTime;
351         sendPlotTelemetry();
352     }
353 }
354 }

```

Listing 4.4: ESP32 Main Program

```

1  #include "BluetoothSerial.h"
2
3  BluetoothSerial SerialBT;
4  #define DEVICE_NAME "ESP32-BT"
5
6  typedef struct __attribute__((packed)) {
7      byte startByte1;
8      byte startByte2;
9      uint16_t leftStickX;
10     uint16_t leftStickY; // Bytes 4-5
11     uint16_t rightStickX; // Bytes 6-7
12     uint16_t rightStickY; // Bytes 8-9
13     uint16_t leftKnob; // Bytes 10-11 (Was byte)
14     uint16_t rightKnob; // Bytes 12-13 (Was byte)
15     byte switches; // Byte 14
16     byte checksum; // Byte 15
17     byte endByte1; // Byte 16 (From Android code)
18     byte endByte2; // Byte 17 (From Android code)
19 }
20 RcPacket;
21
22 typedef union {
23     RcPacket data;
24     byte bytes[sizeof(RcPacket)];
25 }
26 StatePacketUnion;
27
28 StatePacketUnion rcStatePacket;
29 const int STATE_PACKET_SIZE = sizeof(RcPacket);
30 typedef struct __attribute__((packed)) {
31     byte startByte1;
32     byte startByte2;
33     byte eventId;
34     byte checksum;
35 }
36 EventPacket;
37
38 typedef union {
39     EventPacket data;
40     byte bytes[sizeof(EventPacket)];
41 }
42 EventPacketUnion;
43
44 EventPacketUnion rcEventPacket;
45 const int EVENT_PACKET_SIZE = sizeof(EventPacket);
46
47 typedef struct __attribute__((packed)) {
48     byte header1; // 0xCC
49     byte header2; // 0x11
50     uint16_t leftPanelValue;
51     uint16_t rightPanelValue;
52     byte panelStates;
53     byte checksum;
54 }
55 PanelPacket;

```

```

56 const int PANEL_PACKET_SIZE = sizeof(PanelPacket);
57 PanelPacket panelPacket;
58
59 typedef struct __attribute__((packed)) {
60     byte header1; // 0xCC
61     byte header2; // 0x22
62     byte analogValue;
63     byte batteryLevel;
64     byte checksum;
65 }
66 IndicatorPacket;
67 const int INDICATOR_PACKET_SIZE = sizeof(IndicatorPacket);
68 IndicatorPacket indicatorPacket;
69
70 typedef struct __attribute__((packed)) {
71     byte header1; // 0xCC
72     byte header2; // 0x33
73     byte numPlots; // Number of plot values being sent
74     byte plotValue1; // First plot value
75     byte plotValue2; // Second plot value
76     byte plotValue3;
77     byte checksum;
78 }
79 PlotPacket;
80 const int PLOT_PACKET_SIZE = sizeof(PlotPacket);
81 PlotPacket plotPacket;
82
83 // --- START OF NEW CONFIGURATION PACKET LOGIC ---
84
85 // 1. Define the names for our plots that we will send to the Android app.
86 const char * plotNames[] = {
87     "Volts",
88     "Amps",
89     "RPMs"
90 };
91 const int NUM_PLOT_NAMES = sizeof(plotNames) / sizeof(plotNames[0]);
92
93 // 2. Create a function to build and send the variable-length config packet.
94 void sendConfigTelemetry() {
95     byte packetBuffer[128]; // A buffer large enough to hold our headers and
96                             // names
97     int packetIndex = 0;
98
99     // Header for Config Packet: 0xCC 0x44
100     packetBuffer[packetIndex++] = 0xCC;
101     packetBuffer[packetIndex++] = 0x44;
102
103     // Number of names we are sending
104     packetBuffer[packetIndex++] = NUM_PLOT_NAMES;
105
106     // For each name, write its length followed by the name itself
107     for (int i = 0; i < NUM_PLOT_NAMES; i++) {
108         int nameLen = strlen(plotNames[i]);
109         packetBuffer[packetIndex++] = nameLen; // Write the length of the name
110         memcpy( & packetBuffer[packetIndex], plotNames[i], nameLen); // Write the
111                                     // name bytes
112         packetIndex += nameLen;
113     }

```

```

112 // Note: The simple config parser on Android doesn't require a checksum for
113 // this packet.
114
115 if (SerialBT.hasClient()) {
116     SerialBT.write(packetBuffer, packetIndex);
117     Serial.println("Sent Plot Names Configuration Packet.");
118 }
119 // --- END OF NEW CONFIGURATION PACKET LOGIC ---
120
121 unsigned long lastPanelSendTime = 0;
122 const long panelInterval = 1000;
123 unsigned long lastIndicatorSendTime = 0;
124 const long indicatorInterval = 250;
125 unsigned long lastPlotSendTime = 0;
126 const long plotInterval = 50;
127
128 void processAndPrintPacket() {
129     Serial.println("--- Valid 18-Byte State Packet Received! ---");
130
131     Serial.printf("  Sticks L_X: %u, L_Y: %u, R_X: %u, R_Y: %u\n",
132         rcStatePacket.data.leftStickX, rcStatePacket.data.leftStickY,
133         rcStatePacket.data.rightStickX, rcStatePacket.data.rightStickY);
134
135     Serial.printf("  Knobs L: %u, R: %u\n", rcStatePacket.data.leftKnob,
136         rcStatePacket.data.rightKnob);
137
138     bool switch1 = (rcStatePacket.data.switches & (1 << 0)) != 0;
139     bool switch2 = (rcStatePacket.data.switches & (1 << 1)) != 0;
140     bool switch3 = (rcStatePacket.data.switches & (1 << 2)) != 0;
141     bool switch4 = (rcStatePacket.data.switches & (1 << 3)) != 0;
142     bool switch5 = (rcStatePacket.data.switches & (1 << 4)) != 0;
143     bool switch6 = (rcStatePacket.data.switches & (1 << 5)) != 0;
144     Serial.printf("  Switches (Byte: %d): S1=%d, S2=%d, S3=%d, S4=%d, S5=%d, S6
145         =%d\n",
146         rcStatePacket.data.switches, switch1, switch2, switch3, switch4, switch5,
147         switch6);
148     Serial.println("-----");
149 }
150
151 void processEventPacket() {
152     Serial.println("--- Valid 4-Byte Event Packet ---");
153     Serial.printf("  Event ID: 0x%02X\n", rcEventPacket.data.eventId);
154     Serial.println("-----");
155
156     switch (rcEventPacket.data.eventId) {
157     case 0x01:
158         Serial.println("  Action: Top-Left Button Pressed!");
159         break;
160     case 0x02:
161         Serial.println("  Action: Top-Right Button Pressed!");
162         break;
163     case 0x03:
164         Serial.println("  Action: Bottom-Left Button Pressed!");
165         break;
166     case 0x04:
167         Serial.println("  Action: Bottom-Right Button Pressed!");
168         break;
169     }

```

```

166 default:
167     Serial.println("  Action: Unknown Event ID.");
168     break;
169 }
170 }
171
172 // --- START OF SEPARATE SEND FUNCTIONS ---
173
174 void sendPanelTelemetry() {
175     panelPacket.header1 = 0xCC;
176     panelPacket.header2 = 0x11;
177     panelPacket.leftPanelValue = 1234;
178     panelPacket.rightPanelValue = 5678;
179
180     byte states = 0;
181     states |= (1 << 0); // leftOn = true
182     states |= (1 << 1); // rightOn = false
183     states |= (1 << 2); // leftColorIsGreen = true
184     states |= (0 << 3); // rightColorIsGreen = false
185     panelPacket.panelStates = states;
186
187     byte checksum = 0;
188     // Sum payload bytes (from byte 2 up to the checksum byte)
189     for (int i = 2; i < PANEL_PACKET_SIZE - 1; i++) {
190         checksum += ((byte * ) & panelPacket)[i];
191     }
192     panelPacket.checksum = checksum;
193
194     if (SerialBT.hasClient()) {
195         SerialBT.write((byte * ) & panelPacket, PANEL_PACKET_SIZE);
196     }
197 }
198
199 void sendIndicatorTelemetry() {
200     indicatorPacket.header1 = 0xCC;
201     indicatorPacket.header2 = 0x22;
202     indicatorPacket.analogValue = 75; // Example static value
203     indicatorPacket.batteryLevel = 98; // Example static value
204
205     byte checksum = 0;
206     for (int i = 2; i < INDICATOR_PACKET_SIZE - 1; i++) {
207         checksum += ((byte * ) & indicatorPacket)[i];
208     }
209     indicatorPacket.checksum = checksum;
210
211     if (SerialBT.hasClient()) {
212         SerialBT.write((byte * ) & indicatorPacket, INDICATOR_PACKET_SIZE);
213     }
214 }
215
216 void sendPlotTelemetry() {
217     plotPacket.header1 = 0xCC;
218     plotPacket.header2 = 0x33;
219     plotPacket.numPlots = 3;
220
221     float time = millis() / 1000.0 f;
222     plotPacket.plotValue1 = (uint8_t)((((sin(time * 1.5 f * PI) * 0.4 f) + 0.6 f)
        * 255));

```

```

223 plotPacket.plotValue2 = (uint8_t)((((cos(time * 3.0 f * PI) * 0.2 f) + 0.25 f
    ) * 255);
224 plotPacket.plotValue3 = (uint8_t)((((cos(time * 3.0 f * PI) * 0.2 f) + 0.5 f)
    * 255);
225
226 byte checksum = 0;
227 for (int i = 2; i < PLOT_PACKET_SIZE - 1; i++) {
228     checksum += ((byte * ) & plotPacket)[i];
229 }
230 plotPacket.checksum = checksum;
231
232 if (SerialBT.hasClient()) {
233     SerialBT.write((byte * ) & plotPacket, PLOT_PACKET_SIZE);
234 }
235 }
236
237 void setup() {
238     Serial.begin(115200);
239     SerialBT.begin(DEVICE_NAME);
240     Serial.println("ESP32 Bluetooth Receiver Ready.");
241     Serial.printf("Listening for State Packets (%d bytes) and Event Packets (%d
        bytes).\n", STATE_PACKET_SIZE, EVENT_PACKET_SIZE);
242 }
243
244 void loop() {
245     // --- START OF MODIFICATION: Send config on new client connection ---
246     static bool clientIsConnected = false;
247     if (SerialBT.hasClient()) {
248         // If a client has just connected, send the configuration.
249         if (!clientIsConnected) {
250             Serial.println("Client Connected!");
251             clientIsConnected = true;
252             delay(200); // Give the client a moment to be ready before sending data
253             sendConfigTelemetry(); // Send the plot names right after connecting
254         }
255     } else {
256         // If the client has disconnected, reset the flag.
257         if (clientIsConnected) {
258             Serial.println("Client Disconnected.");
259             clientIsConnected = false;
260         }
261     }
262     // --- END OF MODIFICATION ---
263
264     static byte buffer[STATE_PACKET_SIZE * 2];
265     static int bytesRead = 0;
266
267     while (SerialBT.available()) {
268         if (bytesRead < sizeof(buffer)) {
269             buffer[bytesRead++] = SerialBT.read();
270         } else {
271             memmove(buffer, buffer + 1, sizeof(buffer) - 1);
272             bytesRead--;
273             buffer[bytesRead++] = SerialBT.read();
274         }
275     }
276
277     while (true) {

```

```

278 int packetStart = -1;
279 bool isEvent = false;
280 int packetSize = 0;
281
282 for (int i = 0; i <= bytesRead - 2; i++) {
283     if (buffer[i] == 0xAA && buffer[i + 1] == 0x55) {
284         if (bytesRead - i >= STATE_PACKET_SIZE) {
285             packetStart = i;
286             isEvent = false;
287             packetSize = STATE_PACKET_SIZE;
288             break;
289         }
290     }
291     if (buffer[i] == 0xBB && buffer[i + 1] == 0x66) {
292         if (bytesRead - i >= EVENT_PACKET_SIZE) {
293             packetStart = i;
294             isEvent = true;
295             packetSize = EVENT_PACKET_SIZE;
296             break;
297         }
298     }
299 }
300
301 if (packetStart == -1) {
302     break;
303 }
304
305 if (isEvent) {
306     memcpy(rcEventPacket.bytes, & buffer[packetStart], packetSize);
307     byte eventId = rcEventPacket.data.eventId;
308     byte checksum = rcEventPacket.data.checksum;
309     if (eventId == checksum) {
310         processEventPacket();
311     } else {
312         Serial.println("Event packet checksum mismatch!");
313     }
314 } else {
315     memcpy(rcStatePacket.bytes, & buffer[packetStart], packetSize);
316     byte receivedChecksum = rcStatePacket.data.checksum;
317     byte calculatedChecksum = 0;
318     for (int i = 2; i < STATE_PACKET_SIZE - 3; i++) {
319         calculatedChecksum += rcStatePacket.bytes[i];
320     }
321     if (receivedChecksum == calculatedChecksum) {
322         processAndPrintPacket();
323     } else {
324         Serial.printf("State Checksum mismatch! R: %d, C: %d\n",
325             receivedChecksum, calculatedChecksum);
326     }
327 }
328
329 int bytesToRemove = packetStart + packetSize;
330 if (bytesToRemove > bytesRead) {
331     bytesRead = 0;
332 } else {
333     int bytesRemaining = bytesRead - bytesToRemove;
334     memmove(buffer, & buffer[bytesToRemove], bytesRemaining);
335     bytesRead = bytesRemaining;

```

```

335     }
336 }
337
338 unsigned long currentTime = millis();
339
340 if (clientIsConnected) {
341     if (currentTime - lastPanelSendTime >= panelInterval) {
342         lastPanelSendTime = currentTime;
343         sendPanelTelemetry();
344     }
345     if (currentTime - lastIndicatorSendTime >= indicatorInterval) {
346         lastIndicatorSendTime = currentTime;
347         sendIndicatorTelemetry();
348     }
349     if (currentTime - lastPlotSendTime >= plotInterval) {
350         lastPlotSendTime = currentTime;
351         sendPlotTelemetry();
352     }
353 }
354 }

```

Listing 4.5: ESP32 Main Sketch

```

1  #include "BluetoothSerial.h"
2
3  BluetoothSerial SerialBT;
4  #define DEVICE_NAME "ESP32-BT"
5
6  typedef struct __attribute__((packed)) {
7      byte    startByte1;
8      byte    startByte2;
9      uint16_t leftStickX;
10     uint16_t leftStickY;    // Bytes 4-5
11     uint16_t rightStickX;   // Bytes 6-7
12     uint16_t rightStickY;   // Bytes 8-9
13     uint16_t leftKnob;      // Bytes 10-11 (Was byte)
14     uint16_t rightKnob;     // Bytes 12-13 (Was byte)
15     byte    switches;       // Byte 14
16     byte    checksum;       // Byte 15
17     byte    endByte1;       // Byte 16 (From Android code)
18     byte    endByte2;       // Byte 17 (From Android code)
19 } RcPacket;
20
21 typedef union {
22     RcPacket data;
23     byte bytes[sizeof(RcPacket)];
24 } StatePacketUnion;
25
26 StatePacketUnion rcStatePacket;
27 const int STATE_PACKET_SIZE = sizeof(RcPacket);
28 typedef struct __attribute__((packed)) {
29     byte startByte1;
30     byte startByte2;
31     byte eventId;
32     byte checksum;
33 } EventPacket;
34
35 typedef union {

```

```

36   EventPacket data;
37   byte bytes[sizeof(EventPacket)];
38 } EventPacketUnion;
39
40 EventPacketUnion rcEventPacket;
41 const int EVENT_PACKET_SIZE = sizeof(EventPacket);
42
43 typedef struct __attribute__((packed)) {
44     byte      header1;          // 0xCC
45     byte      header2;          // 0x11
46     uint16_t   leftPanelValue;
47     uint16_t   rightPanelValue;
48     byte       panelStates;
49     byte       checksum;
50 } PanelPacket;
51 const int PANEL_PACKET_SIZE = sizeof(PanelPacket);
52 PanelPacket panelPacket;
53
54 typedef struct __attribute__((packed)) {
55     byte      header1;          // 0xCC
56     byte      header2;          // 0x22
57     byte      analogValue;
58     byte      batteryLevel;
59     byte      checksum;
60 } IndicatorPacket;
61 const int INDICATOR_PACKET_SIZE = sizeof(IndicatorPacket);
62 IndicatorPacket indicatorPacket;
63
64 typedef struct __attribute__((packed)) {
65     byte      header1;          // 0xCC
66     byte      header2;          // 0x33
67     byte      numPlots;          // Number of plot values being sent
68     byte      plotValue1;        // First plot value
69     byte      plotValue2;        // Second plot value
70     byte      plotValue3;
71     byte      checksum;
72 } PlotPacket;
73 const int PLOT_PACKET_SIZE = sizeof(PlotPacket);
74 PlotPacket plotPacket;
75
76 // --- START OF NEW CONFIGURATION PACKET LOGIC ---
77
78 // 1. Define the names for our plots that we will send to the Android app.
79 const char* plotNames[] = {"Volts", "Amps", "RPMs"};
80 const int NUM_PLOT_NAMES = sizeof(plotNames) / sizeof(plotNames[0]);
81
82
83 // 2. Create a function to build and send the variable-length config packet.
84 void sendConfigTelemetry() {
85     byte packetBuffer[128]; // A buffer large enough to hold our headers and
86                             names
87     int packetIndex = 0;
88
89     // Header for Config Packet: 0xCC 0x44
90     packetBuffer[packetIndex++] = 0xCC;
91     packetBuffer[packetIndex++] = 0x44;
92
93     // Number of names we are sending

```

```

93     packetBuffer[packetIndex++] = NUM_PLOT_NAMES;
94
95     // For each name, write its length followed by the name itself
96     for (int i = 0; i < NUM_PLOT_NAMES; i++) {
97         int nameLen = strlen(plotNames[i]);
98         packetBuffer[packetIndex++] = nameLen; // Write the length of the name
99         memcpy(&packetBuffer[packetIndex], plotNames[i], nameLen); // Write
100            the name bytes
101         packetIndex += nameLen;
102     }
103     // Note: The simple config parser on Android doesn't require a checksum
104     for this packet.
105
106     if (SerialBT.hasClient()) {
107         SerialBT.write(packetBuffer, packetIndex);
108         Serial.println("Sent Plot Names Configuration Packet.");
109     }
110 }
111 // --- END OF NEW CONFIGURATION PACKET LOGIC ---
112
113 unsigned long lastPanelSendTime = 0;
114 const long panelInterval = 1000;
115 unsigned long lastIndicatorSendTime = 0;
116 const long indicatorInterval = 250;
117 unsigned long lastPlotSendTime = 0;
118 const long plotInterval = 50;
119
120 void processAndPrintPacket() {
121     Serial.println("--- Valid 18-Byte State Packet Received! ---");
122
123     Serial.printf("  Sticks L_X: %u, L_Y: %u, R_X: %u, R_Y: %u\n",
124         rcStatePacket.data.leftStickX, rcStatePacket.data.leftStickY,
125         rcStatePacket.data.rightStickX, rcStatePacket.data.rightStickY
126     );
127
128     Serial.printf("  Knobs L: %u, R: %u\n", rcStatePacket.data.leftKnob,
129         rcStatePacket.data.rightKnob);
130
131     bool switch1 = (rcStatePacket.data.switches & (1 << 0)) != 0;
132     bool switch2 = (rcStatePacket.data.switches & (1 << 1)) != 0;
133     bool switch3 = (rcStatePacket.data.switches & (1 << 2)) != 0;
134     bool switch4 = (rcStatePacket.data.switches & (1 << 3)) != 0;
135     bool switch5 = (rcStatePacket.data.switches & (1 << 4)) != 0;
136     bool switch6 = (rcStatePacket.data.switches & (1 << 5)) != 0;
137     Serial.printf("  Switches (Byte: %d): S1=%d, S2=%d, S3=%d, S4=%d, S5=%d, S6
138         =%d\n",
139         rcStatePacket.data.switches, switch1, switch2, switch3,
140         switch4, switch5, switch6);
141     Serial.println("-----");
142 }
143
144 void processEventPacket() {
145     Serial.println("--- Valid 4-Byte Event Packet ---");
146     Serial.printf("  Event ID: 0x%02X\n", rcEventPacket.data.eventId);
147     Serial.println("-----");
148
149     switch(rcEventPacket.data.eventId) {
150         case 0x01: Serial.println("  Action: Top-Left Button Pressed!"); break;

```

```

145     case 0x02: Serial.println("  Action: Top-Right Button Pressed!"); break;
146     case 0x03: Serial.println("  Action: Bottom-Left Button Pressed!"); break;
147     case 0x04: Serial.println("  Action: Bottom-Right Button Pressed!"); break
148     ;
149     default: Serial.println("  Action: Unknown Event ID."); break;
150 }
151
152 // --- START OF SEPARATE SEND FUNCTIONS ---
153
154 void sendPanelTelemetry() {
155     panelPacket.header1 = 0xCC;
156     panelPacket.header2 = 0x11;
157     panelPacket.leftPanelValue = 1234;
158     panelPacket.rightPanelValue = 5678;
159
160     byte states = 0;
161     states |= (1 << 0); // leftOn = true
162     states |= (1 << 1); // rightOn = false
163     states |= (1 << 2); // leftColorIsGreen = true
164     states |= (0 << 3); // rightColorIsGreen = false
165     panelPacket.panelStates = states;
166
167     byte checksum = 0;
168     // Sum payload bytes (from byte 2 up to the checksum byte)
169     for (int i = 2; i < PANEL_PACKET_SIZE - 1; i++) {
170         checksum += ((byte*)&panelPacket)[i];
171     }
172     panelPacket.checksum = checksum;
173
174     if (SerialBT.hasClient()) {
175         SerialBT.write((byte*)&panelPacket, PANEL_PACKET_SIZE);
176     }
177 }
178
179 void sendIndicatorTelemetry() {
180     indicatorPacket.header1 = 0xCC;
181     indicatorPacket.header2 = 0x22;
182     indicatorPacket.analogValue = 75; // Example static value
183     indicatorPacket.batteryLevel = 98; // Example static value
184
185     byte checksum = 0;
186     for (int i = 2; i < INDICATOR_PACKET_SIZE - 1; i++) {
187         checksum += ((byte*)&indicatorPacket)[i];
188     }
189     indicatorPacket.checksum = checksum;
190
191     if (SerialBT.hasClient()) {
192         SerialBT.write((byte*)&indicatorPacket, INDICATOR_PACKET_SIZE);
193     }
194 }
195
196 void sendPlotTelemetry() {
197     plotPacket.header1 = 0xCC;
198     plotPacket.header2 = 0x33;
199     plotPacket.numPlots = 3;
200
201     float time = millis() / 1000.0f;

```

```

202     plotPacket.plotValue1 = (uint8_t)((((sin(time * 1.5f * PI) * 0.4f) + 0.6f)
203         * 255));
204     plotPacket.plotValue2 = (uint8_t)((((cos(time * 3.0f * PI) * 0.2f) + 0.25f)
205         * 255));
206     plotPacket.plotValue3 = (uint8_t)((((cos(time * 3.0f * PI) * 0.2f) + 0.5f)
207         * 255));
208
209     byte checksum = 0;
210     for (int i = 2; i < PLOT_PACKET_SIZE - 1; i++) {
211         checksum += ((byte*)&plotPacket)[i];
212     }
213     plotPacket.checksum = checksum;
214
215     if (SerialBT.hasClient()) {
216         SerialBT.write((byte*)&plotPacket, PLOT_PACKET_SIZE);
217     }
218 }
219
220 void setup() {
221     Serial.begin(115200);
222     SerialBT.begin(DEVICE_NAME);
223     Serial.println("ESP32 Bluetooth Receiver Ready.");
224     Serial.printf("Listening for State Packets (%d bytes) and Event Packets (%d
225         bytes).\n", STATE_PACKET_SIZE, EVENT_PACKET_SIZE);
226 }
227
228 void loop() {
229     // --- START OF MODIFICATION: Send config on new client connection ---
230     static bool clientIsConnected = false;
231     if (SerialBT.hasClient()) {
232         // If a client has just connected, send the configuration.
233         if (!clientIsConnected) {
234             Serial.println("Client Connected!");
235             clientIsConnected = true;
236             delay(200); // Give the client a moment to be ready before sending data
237             sendConfigTelemetry(); // Send the plot names right after connecting
238         }
239     } else {
240         // If the client has disconnected, reset the flag.
241         if (clientIsConnected) {
242             Serial.println("Client Disconnected.");
243             clientIsConnected = false;
244         }
245     }
246     // --- END OF MODIFICATION ---
247
248     static byte buffer[STATE_PACKET_SIZE * 2];
249     static int bytesRead = 0;
250
251     while (SerialBT.available()) {
252         if (bytesRead < sizeof(buffer)) {
253             buffer[bytesRead++] = SerialBT.read();
254         } else {
255             memmove(buffer, buffer + 1, sizeof(buffer) - 1);
256             bytesRead--;
257             buffer[bytesRead++] = SerialBT.read();
258         }
259     }

```

```

256 }
257
258 while (true) {
259     int packetStart = -1;
260     bool isEvent = false;
261     int packetSize = 0;
262
263     for (int i = 0; i <= bytesRead - 2; i++) {
264         if (buffer[i] == 0xAA && buffer[i + 1] == 0x55) {
265             if (bytesRead - i >= STATE_PACKET_SIZE) {
266                 packetStart = i;
267                 isEvent = false;
268                 packetSize = STATE_PACKET_SIZE;
269                 break;
270             }
271         }
272         if (buffer[i] == 0xBB && buffer[i + 1] == 0x66) {
273             if (bytesRead - i >= EVENT_PACKET_SIZE) {
274                 packetStart = i;
275                 isEvent = true;
276                 packetSize = EVENT_PACKET_SIZE;
277                 break;
278             }
279         }
280     }
281
282     if (packetStart == -1) {
283         break;
284     }
285
286     if (isEvent) {
287         memcpy(rcEventPacket.bytes, &buffer[packetStart], packetSize);
288         byte eventId = rcEventPacket.data.eventId;
289         byte checksum = rcEventPacket.data.checksum;
290         if (eventId == checksum) {
291             processEventPacket();
292         } else {
293             Serial.println("Event packet checksum mismatch!");
294         }
295     } else {
296         memcpy(rcStatePacket.bytes, &buffer[packetStart], packetSize);
297         byte receivedChecksum = rcStatePacket.data.checksum;
298         byte calculatedChecksum = 0;
299         for (int i = 2; i < STATE_PACKET_SIZE - 3; i++) {
300             calculatedChecksum += rcStatePacket.bytes[i];
301         }
302         if (receivedChecksum == calculatedChecksum) {
303             processAndPrintPacket();
304         } else {
305             Serial.printf("State Checksum mismatch! R: %d, C: %d\n",
306                 receivedChecksum, calculatedChecksum);
307         }
308     }
309
310     int bytesToRemove = packetStart + packetSize;
311     if (bytesToRemove > bytesRead) {
312         bytesRead = 0;
313     } else {

```

```

313     int bytesRemaining = bytesRead - bytesToRemove;
314     memmove(buffer, &buffer[bytesToRemove], bytesRemaining);
315     bytesRead = bytesRemaining;
316 }
317 }
318
319 unsigned long currentTime = millis();
320
321 if (clientIsConnected) {
322     if (currentTime - lastPanelSendTime >= panelInterval) {
323         lastPanelSendTime = currentTime;
324         sendPanelTelemetry();
325     }
326     if (currentTime - lastIndicatorSendTime >= indicatorInterval) {
327         lastIndicatorSendTime = currentTime;
328         sendIndicatorTelemetry();
329     }
330     if (currentTime - lastPlotSendTime >= plotInterval) {
331         lastPlotSendTime = currentTime;
332         sendPlotTelemetry();
333     }
334 }
335 }

```

Listing 4.6: ESP32 Main Sketch Bluetooth07.ino

Chapter 5

Conclusion

This system enables low-latency Bluetooth RC control.

Appendix A

Full Source Code

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;
#define DEVICE_NAME "ESP32-BT"

typedef struct __attribute__((packed)) {
    byte startByte1;
    byte startByte2;
    uint16_t leftStickX;
    uint16_t leftStickY; // Bytes 4–5
    uint16_t rightStickX; // Bytes 6–7
    uint16_t rightStickY; // Bytes 8–9
    uint16_t leftKnob; // Bytes 10–11 (Was byte)
    uint16_t rightKnob; // Bytes 12–13 (Was byte)
    byte switches; // Byte 14
    byte checksum; // Byte 15
    byte endByte1; // Byte 16 (From Android code)
    byte endByte2; // Byte 17 (From Android code)
}
RcPacket;

typedef union {
    RcPacket data;
    byte bytes[sizeof(RcPacket)];
}
StatePacketUnion;

StatePacketUnion rcStatePacket;
const int STATE_PACKET_SIZE = sizeof(RcPacket);
typedef struct __attribute__((packed)) {
    byte startByte1;
    byte startByte2;
    byte eventId;
    byte checksum;
}
EventPacket;

typedef union {
    EventPacket data;
    byte bytes[sizeof(EventPacket)];
}
```

```

EventPacketUnion;

EventPacketUnion rcEventPacket;
const int EVENT_PACKET_SIZE = sizeof(EventPacket);

typedef struct __attribute__((packed)) {
byte header1; // 0xCC
byte header2; // 0x11
uint16_t leftPanelValue;
uint16_t rightPanelValue;
byte panelStates;
byte checksum;
}
PanelPacket;
const int PANEL_PACKET_SIZE = sizeof(PanelPacket);
PanelPacket panelPacket;

typedef struct __attribute__((packed)) {
byte header1; // 0xCC
byte header2; // 0x22
byte analogValue;
byte batteryLevel;
byte checksum;
}
IndicatorPacket;
const int INDICATOR_PACKET_SIZE = sizeof(IndicatorPacket);
IndicatorPacket indicatorPacket;

typedef struct __attribute__((packed)) {
byte header1; // 0xCC
byte header2; // 0x33
byte numPlots; // Number of plot values being sent
byte plotValue1; // First plot value
byte plotValue2; // Second plot value
byte plotValue3;
byte checksum;
}
PlotPacket;
const int PLOT_PACKET_SIZE = sizeof(PlotPacket);
PlotPacket plotPacket;

// ——— START OF NEW CONFIGURATION PACKET LOGIC ———

// 1. Define the names for our plots that we will send to the Android app.
const char * plotNames[] = {
"Volts",
"Amps",
"RPMs"
};
const int NUMPLOT_NAMES = sizeof(plotNames) / sizeof(plotNames[0]);

// 2. Create a function to build and send the variable-length config packet.
void sendConfigTelemetry() {
byte packetBuffer[128]; // A buffer large enough to hold our headers and names

```

```

int packetIndex = 0;

// Header for Config Packet: 0xCC 0x44
packetBuffer[packetIndex++] = 0xCC;
packetBuffer[packetIndex++] = 0x44;

// Number of names we are sending
packetBuffer[packetIndex++] = NUMPLOTNAMES;

// For each name, write its length followed by the name itself
for (int i = 0; i < NUMPLOTNAMES; i++) {
    int nameLen = strlen(plotNames[i]);
    packetBuffer[packetIndex++] = nameLen; // Write the length of the name
    memcpy( & packetBuffer[packetIndex], plotNames[i], nameLen); // Write the name by
    packetIndex += nameLen;
}
// Note: The simple config parser on Android doesn't require a checksum for this

if (SerialBT.hasClient()) {
    SerialBT.write(packetBuffer, packetIndex);
    Serial.println("Sent Plot Names Configuration Packet.");
}
}
// ——— END OF NEW CONFIGURATION PACKET LOGIC ———

unsigned long lastPanelSendTime = 0;
const long panelInterval = 1000;
unsigned long lastIndicatorSendTime = 0;
const long indicatorInterval = 250;
unsigned long lastPlotSendTime = 0;
const long plotInterval = 50;

void processAndPrintPacket() {
    Serial.println("—— Valid 18-Byte State Packet Received! ——");

    Serial.printf("  Sticks L_X: %u, L_Y: %u, R_X: %u, R_Y: %u\n",
rcStatePacket.data.leftStickX, rcStatePacket.data.leftStickY,
rcStatePacket.data.rightStickX, rcStatePacket.data.rightStickY);

    Serial.printf("  Knobs L: %u, R: %u\n", rcStatePacket.data.leftKnob, rcStatePacket.data.rightKnob);

    bool switch1 = (rcStatePacket.data.switches & (1 << 0)) != 0;
    bool switch2 = (rcStatePacket.data.switches & (1 << 1)) != 0;
    bool switch3 = (rcStatePacket.data.switches & (1 << 2)) != 0;
    bool switch4 = (rcStatePacket.data.switches & (1 << 3)) != 0;
    bool switch5 = (rcStatePacket.data.switches & (1 << 4)) != 0;
    bool switch6 = (rcStatePacket.data.switches & (1 << 5)) != 0;
    Serial.printf("  Switches (Byte: %d): S1=%d, S2=%d, S3=%d, S4=%d, S5=%d, S6=%d\n",
rcStatePacket.data.switches, switch1, switch2, switch3, switch4, switch5, switch6);
    Serial.println("—————");
}

void processEventPacket() {
    Serial.println("—— Valid 4-Byte Event Packet ——");

```

```

Serial.printf("    Event ID: 0x%02X\n", rcEventPacket.data.eventId);
Serial.println("_____");

switch (rcEventPacket.data.eventId) {
case 0x01:
Serial.println("    Action: Top-Left Button Pressed!");
break;
case 0x02:
Serial.println("    Action: Top-Right Button Pressed!");
break;
case 0x03:
Serial.println("    Action: Bottom-Left Button Pressed!");
break;
case 0x04:
Serial.println("    Action: Bottom-Right Button Pressed!");
break;
default:
Serial.println("    Action: Unknown Event ID.");
break;
}
}

// ——— START OF SEPARATE SEND FUNCTIONS ———

void sendPanelTelemetry() {
panelPacket.header1 = 0xCC;
panelPacket.header2 = 0x11;
panelPacket.leftPanelValue = 1234;
panelPacket.rightPanelValue = 5678;

byte states = 0;
states |= (1 << 0); // leftOn = true
states |= (1 << 1); // rightOn = false
states |= (1 << 2); // leftColorIsGreen = true
states |= (0 << 3); // rightColorIsGreen = false
panelPacket.panelStates = states;

byte checksum = 0;
// Sum payload bytes (from byte 2 up to the checksum byte)
for (int i = 2; i < PANEL_PACKET_SIZE - 1; i++) {
checksum += ((byte * ) & panelPacket)[i];
}
panelPacket.checksum = checksum;

if (SerialBT.hasClient()) {
SerialBT.write((byte * ) & panelPacket, PANEL_PACKET_SIZE);
}
}

void sendIndicatorTelemetry() {
indicatorPacket.header1 = 0xCC;
indicatorPacket.header2 = 0x22;
indicatorPacket.analogValue = 75; // Example static value
indicatorPacket.batteryLevel = 98; // Example static value

```

```

byte checksum = 0;
for (int i = 2; i < INDICATOR_PACKET_SIZE - 1; i++) {
checksum += ((byte * ) & indicatorPacket)[i];
}
indicatorPacket.checksum = checksum;

if (SerialBT.hasClient()) {
SerialBT.write((byte * ) & indicatorPacket , INDICATOR_PACKET_SIZE);
}
}

void sendPlotTelemetry() {
plotPacket.header1 = 0xCC;
plotPacket.header2 = 0x33;
plotPacket.numPlots = 3;

float time = millis() / 1000.0 f;
plotPacket.plotValue1 = (uint8_t)(((sin(time * 1.5 f * PI) * 0.4 f) + 0.6 f) * 255);
plotPacket.plotValue2 = (uint8_t)(((cos(time * 3.0 f * PI) * 0.2 f) + 0.25 f) * 255);
plotPacket.plotValue3 = (uint8_t)(((cos(time * 3.0 f * PI) * 0.2 f) + 0.5 f) * 255);

byte checksum = 0;
for (int i = 2; i < PLOT_PACKET_SIZE - 1; i++) {
checksum += ((byte * ) & plotPacket)[i];
}
plotPacket.checksum = checksum;

if (SerialBT.hasClient()) {
SerialBT.write((byte * ) & plotPacket , PLOT_PACKET_SIZE);
}
}

void setup() {
Serial.begin(115200);
SerialBT.begin(DEVICE_NAME);
Serial.println("ESP32 Bluetooth Receiver Ready.");
Serial.printf("Listening for State Packets (%d bytes) and Event Packets (%d bytes) \n", STATE_PACKET_SIZE, EVENT_PACKET_SIZE);
}

void loop() {
// ——— START OF MODIFICATION: Send config on new client connection ———
static bool clientIsConnected = false;
if (SerialBT.hasClient()) {
// If a client has just connected, send the configuration.
if (!clientIsConnected) {
Serial.println("Client Connected!");
clientIsConnected = true;
delay(200); // Give the client a moment to be ready before sending data
sendConfigTelemetry(); // Send the plot names right after connecting
}
} else {
// If the client has disconnected, reset the flag.
if (clientIsConnected) {

```

```

Serial.println(" Client Disconnected.");
clientIsConnected = false;
}
}
// ——— END OF MODIFICATION ———

static byte buffer[STATE_PACKET_SIZE * 2];
static int bytesRead = 0;

while (SerialBT.available()) {
if (bytesRead < sizeof(buffer)) {
buffer[bytesRead++] = SerialBT.read();
} else {
memmove(buffer, buffer + 1, sizeof(buffer) - 1);
bytesRead--;
buffer[bytesRead++] = SerialBT.read();
}
}

while (true) {
int packetStart = -1;
bool isEvent = false;
int packetSize = 0;

for (int i = 0; i <= bytesRead - 2; i++) {
if (buffer[i] == 0xAA && buffer[i + 1] == 0x55) {
if (bytesRead - i >= STATE_PACKET_SIZE) {
packetStart = i;
isEvent = false;
packetSize = STATE_PACKET_SIZE;
break;
}
}
if (buffer[i] == 0xBB && buffer[i + 1] == 0x66) {
if (bytesRead - i >= EVENT_PACKET_SIZE) {
packetStart = i;
isEvent = true;
packetSize = EVENT_PACKET_SIZE;
break;
}
}
}

if (packetStart == -1) {
break;
}

if (isEvent) {
memcpy(rcEventPacket.bytes, & buffer[packetStart], packetSize);
byte eventId = rcEventPacket.data.eventId;
byte checksum = rcEventPacket.data.checksum;
if (eventId == checksum) {
processEventPacket();
} else {

```

```

Serial.println("Event packet checksum mismatch!");
}
} else {
memcpy(rcStatePacket.bytes, & buffer[packetStart], packetSize);
byte receivedChecksum = rcStatePacket.data.checksum;
byte calculatedChecksum = 0;
for (int i = 2; i < STATE_PACKET_SIZE - 3; i++) {
calculatedChecksum += rcStatePacket.bytes[i];
}
if (receivedChecksum == calculatedChecksum) {
processAndPrintPacket();
} else {
Serial.printf("State Checksum mismatch! R: %d, C: %d\n", receivedChecksum, calculatedChecksum);
}
}

int bytesToRemove = packetStart + packetSize;
if (bytesToRemove > bytesRead) {
bytesRead = 0;
} else {
int bytesRemaining = bytesRead - bytesToRemove;
memmove(buffer, & buffer[bytesToRemove], bytesRemaining);
bytesRead = bytesRemaining;
}
}

unsigned long currentTime = millis();

if (clientIsConnected) {
if (currentTime - lastPanelSendTime >= panelInterval) {
lastPanelSendTime = currentTime;
sendPanelTelemetry();
}
if (currentTime - lastIndicatorSendTime >= indicatorInterval) {
lastIndicatorSendTime = currentTime;
sendIndicatorTelemetry();
}
if (currentTime - lastPlotSendTime >= plotInterval) {
lastPlotSendTime = currentTime;
sendPlotTelemetry();
}
}
}
}

```